

# Sentinelleai — Audit Report

Faucet.sol · f1ef5c85-ef6f-43b1-b7df-3df214096b10 · 2026-05-22 20 h 47 min 39 s

## CAUTION 62 / 100 8 consolidated findings

The Faucet contract is a straightforward ETH dispenser with several design-level risks but no immediately exploitable critical vulnerability in its current form. The static-sentinel REE-1 flag (SC08) is a false positive: state is correctly updated before the external call and both public entry points are protected by OpenZeppelin's nonReentrant modifier — this is confirmed by four of five agents (attacker, economist, historian, formalist-context). The dominant risks are: (1) SC01 — single-EOA owner can drain all funds via withdrawAll or manipulate parameters without timelock, representing a centralization risk; (2) SC03 — Sybil drain through unlimited fresh recipient addresses via the permissionless dripTo function, which also enables cooldown griefing against legitimate users; (3) SC04 — unbounded parameter setting (dripAmount=0 wastes cooldowns, cooldownTime=0 removes rate limiting). The withdrawAll function also lacks the nonReentrant modifier applied to drip functions, creating an inconsistency. No reentrancy, flash-loan, oracle, or upgrade surface vulnerabilities are present. The contract is suitable for low-value testnet use but requires access-control hardening (multisig + timelock), Sybil resistance, and parameter bounds before deployment in any higher-value context.

## Severity breakdown

2 HIGH 2 MEDIUM 3 LOW 1 INFO

## Static sentinel pre-scan

Static sentinels matched 1 pattern(s) — 1 HIGH.

**HIGH** SC08 · REE-1 · line 49

### External call before state mutation (classic reentrancy)

Pattern call.value() / .call{value:}() followed by state changes — checks-effects-interactions violated.

```
(bool sent, ) = recipient.call{value: amount}("");
```

## Consolidated findings

**HIGH** CVSS 7.5 SC01 – Access Control

### Centralized Single-Owner Control with No Timelock Can Drain All Funds

The contract uses a single owner (Ownable2Step) with unrestricted ability to call `withdrawAll()` to transfer the entire native balance to themselves, `setDripAmount()` to any value including 0 or the full balance, and `setCooldownTime()` to 0 to remove rate limiting. If the owner EOA is compromised or acts maliciously, all faucet funds can be drained in a single transaction with no delay or community override. Ownable2Step mitigates accidental transfer but does not protect against a compromised key.

Faucet.sol:withdrawAll() lines 66-72; setDripAmount() lines 56-59;  
setCooldownTime() lines 61-64

→ Transfer ownership to a multisig (e.g., Gnosis Safe). Implement a timelock (minimum 24 hours) for parameter changes and withdrawals. Add hard caps: `require(_amount > 0 && _amount <= maxDripAmount)` and `require(_seconds >= minCooldown)`. Consider an emergency pause rather than unrestricted withdrawal.

Confirmed by: attacker, developer

**HIGH** CVSS 7.2 SC03 – Logic / Economic Design

### Sybil Drain: Per-Recipient Cooldown Does Not Prevent Multi-Address Emptying

Cooldown is keyed by recipient address. The permissionless `dripTo(address payable)` function allows any caller to request funds for any address. An attacker can generate N fresh addresses and call `dripTo(address_i)` for each, bypassing the cooldown entirely since `lastDripTime` for each fresh address is 0. This can drain the entire faucet balance in a single block, denying service to all legitimate users. The attack cost is only gas.

Faucet.sol:dripTo() lines 37-39; \_drip() lines 42-52

→ Implement Sybil resistance: signed vouchers (EIP-712), allowlists, proof-of-personhood, or off-chain CAPTCHA-backed issuance. If `dripTo` is not required, remove it or restrict it to trusted relayers. Add per-epoch budget caps.

Confirmed by: attacker, economist

**MEDIUM** CVSS 5.3 SC10 – Denial of Service / Logic

### dripTo Enables Cooldown Griefing Against Legitimate Users

Any caller can invoke dripTo(victim) to trigger a drip to a victim address and set lastDripTime[victim] to the current timestamp. This resets the victim's cooldown without their consent. A griever can monitor known user addresses and front-run their claims, forcing them to wait another full cooldown period. The attacker only pays gas and the victim receives the drip amount (so it is not a fund-theft attack), but it disrupts user experience and can be used to manipulate timing for integrations.

Faucet.sol:dripTo() lines 37–39; \_drip() cooldown update line 46

→ Require recipient consent via an EIP-712 signature (recipient, amount, nonce, deadline, chainId) verified on-chain before executing dripTo. Alternatively remove dripTo or restrict it to a trusted relayer role.

Confirmed by: attacker, economist

**MEDIUM** CVSS 5.0 SC04 – Parameter Validation

### Unbounded Owner Parameters Can Brick Faucet or Remove Rate Limiting

setDripAmount accepts any uint256 including 0. Setting dripAmount to 0 causes drip calls to succeed (balance check passes, cooldown is consumed) while transferring 0 wei, wasting users' cooldown windows. Setting it to a value exceeding the balance makes all claims revert. setCooldownTime accepts 0, removing all rate limiting and enabling unlimited sequential claims until the faucet is empty. These are exploitable by a compromised or malicious owner and also represent misconfiguration risks.

Faucet.sol:setDripAmount() lines 56–59; setCooldownTime() lines 61–64

→ Add validation: require(\_amount > 0, 'dripAmount must be > 0'); require(\_amount <= hardCap); require(\_seconds >= minCooldown). Emit events before applying changes and enforce a timelock.

Confirmed by: attacker, formalist

**LOW** CVSS 3.1 SC08 – Reentrancy

### withdrawAll Lacks nonReentrant Modifier — Inconsistent Protection

The withdrawAll function sends the full native balance to msg.sender via a low-level call without the nonReentrant modifier applied to drip and dripTo. While a reentrant call would find the balance already zero (making exploitation unlikely in practice), the inconsistency violates the principle of uniform defense-in-depth and could become exploitable if the function is modified in future versions.

Faucet.sol:withdrawAll() lines 66–72

→ Add the nonReentrant modifier to withdrawAll for consistency and defense-in-depth.

Confirmed by: economist

**LOW** CVSS 2.8 SC10 – Logic

### **withdrawAll Sends to msg.sender — Fails If Owner Is a Non-Payable Contract**

withdrawAll sends native currency to msg.sender. If ownership is transferred to a contract wallet whose receive/fallback rejects ETH or reverts, withdrawAll will always revert and funds become inaccessible through this path. This is a latent risk that materializes upon ownership transfer to certain governance or multisig contracts.

Faucet.sol:withdrawAll() lines 66-72

→ Accept an explicit payable recipient parameter: withdrawAll(address payable to). Validate to != address(0) and document that the recipient must be able to receive native currency.

Confirmed by: attacker

**LOW** CVSS 2.1 SC03 – Timestamp Dependence

### **block.timestamp Dependency for Cooldown Allows Minor Validator Manipulation**

The cooldown check uses block.timestamp. Validators can influence timestamps within consensus-allowed bounds (typically  $\pm 15$  seconds on Ethereum, larger on some L2s). This allows a validator-colluding attacker to claim marginally earlier than strict wall-clock time permits. For a daily faucet this is a negligible risk but worth noting.

Faucet.sol:\_drip() line 44

→ For a daily faucet, timestamp use is acceptable. If strict timing is required, use block numbers with conservative conversion or off-chain signed claim windows.

Confirmed by: attacker

**INFO** CVSS 0.0 SC08 – Reentrancy

### **Static-Sentinel REE-1 Refuted: \_drip External Call Is Not Classic Reentrancy**

The deterministic pre-scan flagged line 49 as an external call before state mutation. In the actual code, lastDripTime[recipient] is written at line 46 before the external call at line 49, correctly following checks-effects-interactions. Both public entry points (drip and dripTo) are protected by OpenZeppelin's nonReentrant modifier. A reentrant call would fail on the mutex and, even without it, the cooldown state is already committed. The Drip event emitted after the call carries no state relied upon by the contract. This is a false positive.

Faucet.sol:\_drip(), lines 43-52

→ No action required. Suppress or annotate the static rule for this location. Retain nonReentrant and the current CEI ordering as defense-in-depth.

Confirmed by: attacker, economist, historian

## Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

**CRITICAL** **reentrancy** ethereum 2022-04-30 **\$80 000 000**

Rari Capital/Fei Protocol — Flashloan Attack + Reentrancy (2022-04-30)

<https://certik.medium.com/fei-protocol-incident-analysis-8527440696cc>

**CRITICAL** **reentrancy** fantom 2021-12-18 **\$30 000 000**

Grim Finance — Flashloan & Reentrancy (2021-12-18)

<https://cointelegraph.com/news/defi-protocol-grim-finance-lost-30m-in-5x-reentrancy-hack>

**CRITICAL** **reentrancy** ethereum 2020-04-19 **\$25 000 000**

LendfMe — ERC777 Reentrancy (2020-04-19)

<https://peckshield.medium.com/uniswap-lendf-me-hacks-root-cause-and-loss-analysis-50f3263dcc09>

**MEDIUM** **bm25** 2022-11-01

stakehouse — Reentrancy - State access after external call

<https://github.com/code-423n4/2022-11-stakehouse-findings/issues/50>

**HIGH** **bm25** 2023-10-01

zksync — L1WethBridge.sol#L233 : possibility of reentrancy attack due to the state update after external call

<https://github.com/code-423n4/2023-10-zksync-findings/issues/779>

**HIGH** **bm25** 2023-09-01

ondo — Stealing extra mint fund by applying reentrancy attack on `_execute` with calling `approve()` again due to external call before crucial state update

<https://github.com/code-423n4/2023-09-ondo-findings/issues/414>

**HIGH** **bm25** 2023-03-01

asymmetry — An attacker can call back into the ``withdraw()`` function before it completes leading to reentrancy attacks.

<https://github.com/code-423n4/2023-03-asymmetry-findings/issues/393>

**HIGH** **bm25** 2024-09-01

kakarot — Unchecked External Call Success in `execute_starknet_call` and `exec_precompile` May Lead to Unexpected Behavior and Reentrancy Attacks

<https://github.com/code-423n4/2024-09-kakarot-findings/issues/115>

**MEDIUM** **bm25** 2022-05-01

rubicon — No check for external call in ```tailoff``` as such malicious strategists could call any external contract from any address

<https://github.com/code-423n4/2022-05-rubicon-findings/issues/144>

**HIGH** bm25 2023-07-01

tapioca — External contracts can intentionally call the burn function and trigger reentrant attacks.

<https://github.com/code-423n4/2023-07-tapioca-findings/issues/940>

**HIGH** bm25 ethereum 2023-10-31

UniBotRouter — Arbitrary External Call (2023-10-31)

<https://twitter.com/PeckShieldAlert/status/1719251390319796477>

**HIGH** bm25 ethereum 2023-10-24

MaestroRouter2 — Arbitrary External Call (2023-10-24)

[https://twitter.com/Phalcon\\_xyz/status/1717014871836098663](https://twitter.com/Phalcon_xyz/status/1717014871836098663)

---

Generated by Sentinelleai on 2026-05-23T00:50:27.767Z. Audit ID: f1ef5c85-ef6f-43b1-b7df-3df214096b10.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.